Bridging the GPU Utilization Gap: Predictive Multi-Dimensional Resource Scheduling for AI Workloads

Yilei Lu
lu-yl20@mails.tsinghua.edu.cn
Tsinghua University and Baihai
Beijing, China

Dongbiao He*
hdb@seu.edu.cn
Southeast University
Nanjing, China

Teng Ma
sima.mt@alibaba-inc.com
Alibaba Group
Beijing, China

Zhe Liu
lz@baihai.ai
Baihai
Beijing, China

Letian Ruan
1291903308rlt@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Jinlei Jiang
jjlei@tsinghua.edu.cn
Tsinghua University
Beijing, China

Yongwei Wu
wuyw@tsinghua.edu.cn
Tsinghua University
Beijing, China

Abstract

Modern AI data centers face a critical paradox: while machine learning workloads dominate infrastructure demands, actual GPU utilization remains consistently low. Existing schedulers fail to coordinate heterogeneous resources effectively, lack predictive capabilities for dynamic workloads, and cannot balance isolation requirements with sharing optimization in multi-tenant clusters. This paper presents Wind, a novel resource scheduler that bridges the GPU utilization gap through predictive scheduling and geometric resource coordination. Wind introduces three key innovations: (1) a resource prediction framework that leverages historical execution patterns to forecast task requirements and completion times with high accuracy;(2) a unified scheduling architecture supporting isolation, sharing, preemption, and prioritization policies that eliminate resource fragmentation while maintaining performance guarantees; and (3) a Hilbert curve-based multi-dimensional scheduling algorithm that maps CPU-memory-GPU resource space to preserve spatial locality while achieving linear computational complexity.

Our evaluation on a 139-node cluster demonstrates that Wind consistently outperforms K8s Default, DRF, and Synergy across diverse workloads. It reduces average response time by 33–48%, lowers P99 JCT by 37%, and improves throughput by up to 46.6% under bursty loads. GPU utilization remains above 92%, while medium-load scenarios yield upmei to 25% resource savings, highlighting the practicality of Wind for production-scale AI clusters.

CCS Concepts: • **Software and its engineering → Scheduling**; • **Computing methodologies → Machine learning**.

Keywords: GPU resource scheduling, AI workloads , Predictive modeling , Hilbert curve

ACM Reference Format:
Yilei Lu, Dongbiao He, Teng Ma, Zhe Liu, Letian Ruan, Jinlei Jiang, and Yongwei Wu. 2026. Bridging the GPU Utilization Gap: Predictive Multi-Dimensional Resource Scheduling for AI Workloads. In *21st European Conference on Computer Systems (EUROSYS '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3767295.3803579

*Corresponding author (hdb@seu.edu.cn).

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland Uk

ACM ISBN 979-8-4007-2212-7/2026/04.
https://doi.org/10.1145/3767295.3803579

1 Introduction

In recent years, artificial intelligence and machine learning workloads have emerged as core components of modern data centers [11, 41, 48, 50]. According to McKinsey's analysis, approximately 70% of data center infrastructure requirements are dedicated to machine learning training and inference tasks [34]. Data center clusters are managed in a multi-tenant fashion, providing services to diverse user groups based on their specific requirements while incorporating resource regulation and access control mechanisms. However, empirical studies [63] reveal that GPU utilization for individual tasks remains suboptimal, with overall GPU resource utilization averaging below 50% [15]. This paradox between surging resource demands and low utilization

rates exposes fundamental limitations in traditional resource scheduling systems: simplistic static allocation strategies (e.g., Kubernetes' static bin-packing algorithms [45]) fail to effectively accommodate the dynamic, heterogeneous, and multi-tenant isolation requirements inherent in machine learning workloads [3]. These data centers encompass substantial heterogeneous computing resources that host numerous AI workloads, which require efficient schedulers to coordinate resource allocation and workload distribution. Such schedulers [49, 63] are essential to ensure efficient AI task execution [30, 37], optimal hardware resource utilization [32, 65], and achievement of other scheduling objectives [51, 59].

Existing scheduling systems confront three fundamental challenges in resource orchestration:**(1) Inadequate multi-dimensional resource coordination**, which results in a failure to achieve synergistic allocation across heterogeneous resource types including CPU, memory, and GPU components. Contemporary schedulers, exemplified by Kubernetes' default scheduler, treat computational resources as orthogonal dimensions, neglecting the intrinsic dependencies between GPU memory bandwidth and compute throughput characteristics, resulting in suboptimal resource placement decisions [42]. **(2) Limited predictive modeling capabilities** arise in conventional scheduling algorithms when confronted with dynamic workload patterns, preventing an accurate estimation of task resource requirements and execution characteristics based on historical profiling data [2, 24, 57, 58]. Traditional resource management frameworks rely predominantly on user-specified resource declarations, whereas actual demands of machine learning workloads exhibit significant variability due to evolving data distributions and model architectural changes. Production-scale analyses reveal that a substantial proportion of deep learning training workloads demonstrate considerable deviation between the declared and actual resource consumption patterns [40, 44, 47]. **(3) The fundamental tension between resource isolation and sharing optimization:** existing systems lack sophisticated mechanisms to balance resource exclusivity guarantees with utilization efficiency in multi-tenant environments. Strict isolation [21] approaches provide deterministic performance guarantees but introduce resource fragmentation overhead, while permissive sharing strategies [29] enable a higher utilization density at the cost of performance interference. Cloud infrastructure studies [54, 56] indicate that inference workloads that execute on shared GPU resources may experience significant tail latency degradation compared to dedicated resource allocation scenarios.

To address these challenges, this paper proposes a resource scheduling system named Wind. The core contributions encompass the following four aspects:

1) We analyze **the diversity of AI workloads**, revealing high burstiness and heterogeneity in their demands across compute, storage, and network resources. By quantitatively comparing GPU sharing versus isolation, we derive key insights for designing dynamic schedulers in AI environments.

2) We propose a **history-based task parameter prediction method** that accurately predicts task processing time and resource requirements. This method matches similar tasks from a historical task database, then uses logistic regression models to learn the mapping relationship between task attributes and execution parameters, providing crucial insights for scheduling decisions.

3) We present a **fine-grained scheduling mechanism** that addresses multi-priority AI workload scheduling through dynamic resource quotas and preemption-aware policies. It provides elastic resource boundaries for burstable tasks and GPU time-slicing with proactive release, enabling precise allocation and efficient multi-task sharing.

4) We develop a **Hilbert-mapping-based scheduler for multi-dimensional resources.** By projecting compact 3–4D task and node descriptors onto a 1D Hilbert curve, the model enables unified similarity measurement and efficient matching. The design incorporates priority-aware Hilbert spaces with tiered distance thresholds and a three-stage scheduling pipeline, ensuring responsiveness for high-priority tasks while maintaining overall resource utilization.

Our 139-node cluster evaluation shows that Wind consistently outperforms Kubernetes' default scheduler, DRF, and Synergy across heterogeneous workloads. It reduces average response time by up to 44.4%, improves system throughput by 46.6%, and lowers burst-task latency by 47.9%. For training, Wind achieves the highest throughput (1.42 min/job) and GPU utilization (91.2%), while for inference it cuts P99 JCT by 37% over Synergy. Moreover, it sustains >92% GPU utilization and yields up to 25% resource savings under medium loads, demonstrating its ability to balance diverse priorities with strong service quality guarantees.

Table 1. Different types of machines in the GPU cluster.

Systems	CPUs	Mem(GB)	GPUs	GPU type	Nodes
IDP	128	1024	8	A800	68
	40	256	2	T4	12
	96	1024	4	V100	24
	64	256	8	In-house	31
	192	2048	8	H800	4

Table 2. Distribution of task types and runtime statistics.

Task Type	Count	%	Avg (min)	Min (min)	Max (min)
Short tasks[1]	809	77%	23	17	30
Medium tasks[2]	125	12%	135	35	276
Long Services[3]	113	11%	413	362	728
Total	**1047**	**100%**	–	–	–

[1]Scripts and small batch processes (runtime $\leq$30 min)
[2]Training tasks (runtime 30 min to 6 hours)
[3]Continuous services (runtime $\geq$6 hours)

2 Background

This paper focuses on the task scheduling problem of deep neural network (DNN) models (including CNN [4], Transformer [19], and Transformer-based generative models [55]) on heterogeneous GPU clusters. To begin, we present a brief overview of IDP (Intelligent Development Platform) developed by Baihai Technology, highlighting its core software and hardware components that form the foundational infrastructure for conducting and executing the framework of Wind. Following this, we uncover the key characteristics of AI workloads and the significant challenges that arise in the context of task scheduling. As a result, it is imperative for us to develop a novel and efficient scheduling framework that can effectively address these challenges.

2.1 The Overview of Baihai IDP

Hardware Architecture: Table 1 provides a detailed comparison of different machine types within a GPU cluster, showcasing their capabilities in terms of CPUs, memory (GB), number and type of GPUs, and the number of nodes. There are a total of 139 nodes, including 68 A800 nodes and 4 H800 nodes. The diversity and scale of these configurations highlight Wind's capacity to support extensive research and application in large-scale GPU scheduling systems. The varied setups allow for testing different scenarios, optimizing performance, and adjusting to the specific demands of advanced computational tasks.

Software Architecture: Wind presents a comprehensive four-tier architecture designed to address the complex challenges of heterogeneous GPU cluster management and AI workload orchestration. The platform integrates an AI Platform layer featuring IDP Studio for streamlined MLOps workflows, IDP LM for zero-code operations, and IDP MaaS for seamless model marketplace access. Built upon versatile AI Algorithm Frameworks that support TensorFlow [39], PyTorch [26], Ray [36], and XGBoost [9], the system provides critical flexibility for diverse computational workloads. The Wind platform implements task scheduling to optimize AI workload performance through dynamic resource adaptation, while providing compute resource management that orchestrates heterogeneous GPU, TPU, and CPU resources with multi-tenant isolation, continuous monitoring, and maintenance capabilities. This unified architecture effectively bridges the gap between complex resource heterogeneity and the demanding requirements of modern AI workloads, establishing a foundation for efficient, scalable, and reliable AI infrastructure deployment.

2.2 Workload Diversity Characteristics

The workloads within AI environments exhibit significant heterogeneity, presenting unique challenges for AI system

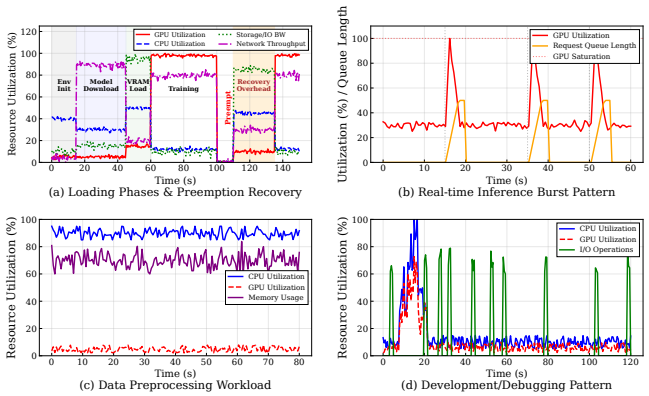


Figure 1. Task burstiness and resource contention.

architecture. Based on our empirical data collection and analysis, we observe the following distinctive patterns in task distribution:

(1)Long-Tail Distribution of Tasks: Table 2 reveals pronounced asymmetry in task distribution patterns. Short-duration tasks constitute an overwhelming majority (77%) of all tasks, with a mean execution time of merely 23 minutes, representing typical "rapid experimentation" workloads. In contrast, medium-duration tasks, while comprising only 12% of tasks, demonstrate substantially longer average execution times (135 minutes) with considerable runtime variance (35-276 minutes). Most notably, long-running service tasks, despite representing just 11% of the total task count, exhibit extraordinarily extended average runtimes of 413 minutes, with maximum continuous execution reaching 728 minutes (approximately 12 hours). **(2)Resource Utilization Asymmetry:** Although short-duration tasks numerically dominate the workload, their cumulative resource consumption remains relatively limited. Conversely, the small fraction of long-running service tasks potentially monopolizes a disproportionate share of system resources. Our analysis suggests that, measured by average runtime, the 11% of long-running service tasks may consume over 50% of system resource time. **(3)High Variance in Task Duration:** The empirical data demonstrates substantial variance in execution times across different task categories. Notably, even within individual categories, task durations exhibit considerable internal variance. Medium-duration tasks exemplify this phenomenon, with the ratio between minimum and maximum runtimes approaching 1:8 (35 minutes versus 276 minutes). *This high-variance characteristic substantially complicates resource pre-allocation strategies and task scheduling algorithms.*

2.3 Task Burstiness and Resource Contention

Task Burstiness. AI workloads exhibit pronounced temporal locality and highly non-linear resource demand patterns that defy conventional capacity planning assumptions. As

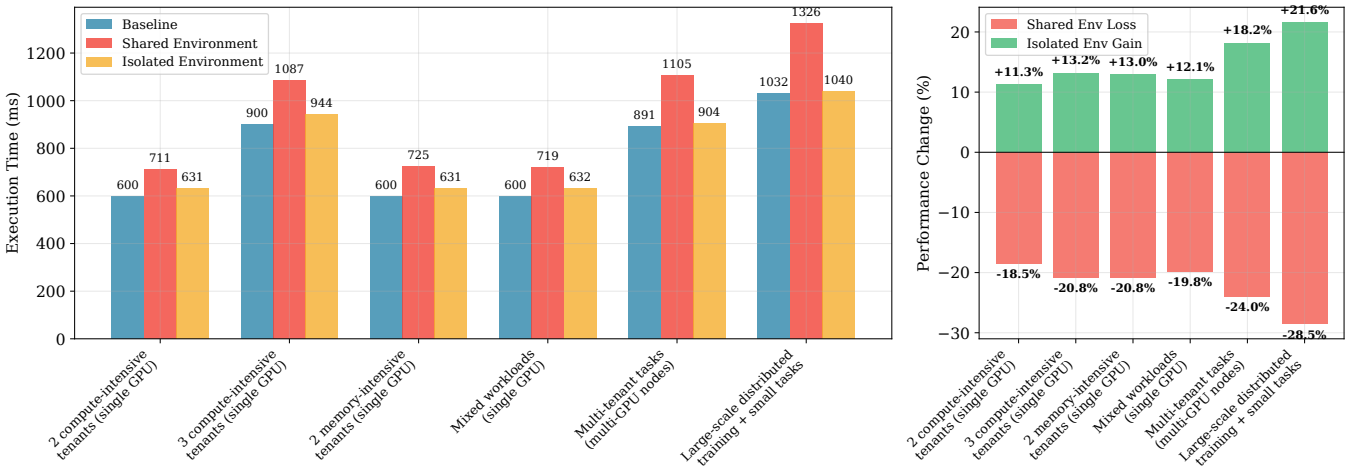


Figure 2. Performance analysis of GPU resource sharing in various multi-tenant scenarios.

illustrated in Figure 1(a), training workflows exhibit fine-grained phase transitions. Initial loading stages including environment setup, model download, and VRAM loading consume peak network and storage bandwidth. While GPU utilization [1] stabilizes above 95% during active training, preemption events trigger substantial recovery overheads. These overheads manifest as prolonged I/O spikes and execution delays required for state restoration. Figure 1(b) reveals the pulsatile nature of real-time inference workloads, where GPU utilization can spike from a 30% idle state to full saturation within seconds. Such high-frequency, high-amplitude resource demand oscillations render conventional history-based prediction algorithms ineffective, exposing a critical gap in existing resource forecasting methodologies. The development and debugging patterns shown in Figure 1(d), while exhibiting relatively modest overall resource consumption, introduce unpredictable intermittent burst behaviors that further complicate task scheduling approaches.

Resource Contention. The heterogeneous nature of AI task types creates structural disparities in compute, storage, and network resource preferences, culminating in multi-layered resource competition scenarios. Figure 1(c) demonstrates that data preprocessing and similar CPU-intensive operations monopolize 90% of available CPU capacity while maintaining minimal GPU requirements (2-8%), ostensibly complementing the resource profiles of GPU-intensive training and inference tasks. However, in practice, multi-tenant environments transform this apparent resource complementarity into fierce contention due to suboptimal scheduling policies. The situation becomes particularly acute when inference tasks encounter sudden request surges—scheduling latencies trigger request queue buildup, precipitating cascading performance degradation and resource waste. Moreover, the temporal overlap between high I/O demands during model loading phases and massive distributed communication requirements during training execution exacerbates contention intensity across network and storage subsystems.

[1]GPU utilization (%) is measured via NVIDIA DCGM, primarily using `DCGM_FI_PROF_SM_ACTIVE` (SM active time) or `DCGM_FI_DEV_GPU_UTIL`, with VRAM usage from `DCGM_FI_DEV_FB_USED`.

2.4 GPU Resource Sharing vs Isolation

GPU resource sharing [46, 60, 66] has emerged as a prevalent strategy to enhance computational resource utilization efficiency. However, this sharing paradigm introduces some performance challenges and resource contention issues in multi-tenant environments. While prior research has investigated GPU virtualization techniques and resource isolation mechanisms, there remains a notable absence of systematic analysis regarding the impact of GPU sharing across diverse workload characteristics and varying tenant scales.

Our preliminary experiments reveal substantial performance degradation resulting from GPU resource sharing in multi-tenant scenarios (shown in Figure 2). In single-GPU configurations, even with merely two compute-intensive tenants coexisting, we observed an 18.50% performance deterioration; this degradation intensifies to 20.80% when the number of tenants increases to three. Memory-intensive workloads similarly experience considerable resource contention, manifested as a 20.80% performance reduction. In mixed workload scenarios, which more accurately reflect production environments, performance decreased by 19.80%.

The situation becomes increasingly critical in more complex multi-GPU environments. Multi-tenant workloads executing across multiple GPU nodes suffer a 24.00% performance penalty, while the co-location of large-scale distributed training with smaller tasks results in performance degradation reaching 28.50%. These findings indicate that resource contention issues exhibit non-linear growth patterns as system complexity and tenant heterogeneity increase. Significantly, our experiments demonstrate that these performance issues can be substantially mitigated through appropriate

resource isolation mechanisms. Across various scenarios, isolation techniques enabled performance improvements ranging from 11.30% to 21.60%, with particularly pronounced benefits observed in complex multi-GPU and heterogeneous workload environments. In practice, we enforce strict host memory isolation via Kubernetes `cgroups` and guide VRAM quotas and limits using the aforementioned predictions, preventing long-running jobs from inefficiently monopolizing or over-reserving GPU memory and further reducing on-node contention.

While resource isolation mechanisms effectively mitigate performance contention issues in multi-tenant environments, their implementation introduces a series of adverse effects, primarily manifested as reduced resource utilization and exacerbated fragmentation problems. Our experimental data reveals that under shared mode, resource fragmentation remains relatively low, with fragmentation rates maintained within the 15%-35%. However, when strict resource isolation policies are enabled, fragmentation issues significantly deteriorate, with fragmentation rates escalating to 35%-50%, representing an average increase of approximately 67%.

This fragmentation intensification primarily stems from isolation mechanisms' pre-allocation and static partitioning of resources, preventing flexible resource sharing and dynamic adjustment among tenants. Even when certain tenants' actual resource demands fall below their allocated quotas, other tenants cannot access these idle resources, consequently resulting in resource wastage. In `Baihai` IDP, single-GPU scenarios currently rely on sequential task switching and CRIU-based preemption. However, the recovery overhead increases approximately linearly with the VRAM footprint, making preemption costly for high-memory jobs. Consequently, runtime memory pressure must still be handled at the application level, underscoring the need for more dynamic resource orchestration and finer-grained VRAM reuse and reclamation.

3 Design of Wind

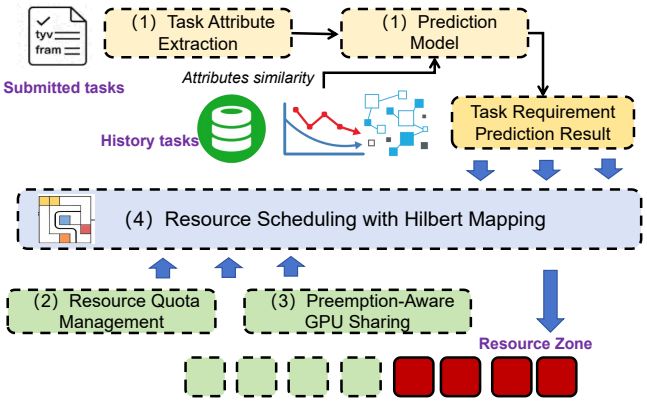


Figure 3. The overall architecture of Wind.

Table 3. Taxonomy of task attributes in Wind.

Category	Specific Attributes
Semantic	Model Architecture (e.g., LLM, ViT), Framework Type, Model Size, Dataset Type/Scale
Structural	GPU Vendor/Model/Count, VRAM Capacity, Priority Class, Est. Duration, Preemptibility
Phase Markers	Model Loading, Tokenization, Quantization, LoRA Fine-tuning, Training/Inference, Model Merging

Wind is a unified prediction-aware scheduling framework designed to improve resource utilization and scheduling efficiency in large-scale, multi-tenant AI computing environments. Figure 3 illustrates the architecture of Wind, which consists of four major functional components: Task Understanding and Prediction, Resource Quota Management, Preemption-Aware GPU Sharing, and Resource Scheduling with Hilbert Mapping [6, 25, 61]. Together, these components support critical scheduling policies such as isolation, sharing, preemption, and prioritization.

1) Task Attribute Extraction and Prediction Module *(Details in §4)*: This module serves as the predictive layer, extracting semantic and structural attributes (shown in Table3) from incoming tasks and using machine learning models, trained on historical execution traces, to predict resource requirements and expected processing times. By transforming reactive scheduling into proactive resource planning, it enables the system to make informed allocation decisions prior to task execution.

This module underpins *prioritization* by producing accurate estimates of task completion times and resource consumption, which downstream scheduling algorithms leverage to rank and schedule tasks effectively. Moreover, it enables *isolation* by providing these precise resource predictions to the Resource Quota Management module, which then validates them against team-specific quotas to enforce resource boundaries.

2) Resource Quota Management *(Details in §5.1)*: This module enforces hierarchical resource governance by managing team-level resource allocations through strict policies, while also providing controlled mechanisms for resource flexibility. It continuously monitors resource utilization, tracks allocation patterns, and enforces policy constraints across organizational boundaries.

As the primary enforcement mechanism for *isolation*, it guarantees that each team operates within specified resource boundaries. Simultaneously, it facilitates *sharing* by supporting resource pooling and borrowing protocols. When a team's quota is insufficient for an incoming task, this module can trigger two actions: attempt to borrow resources from other teams, or interface with an external cluster autoscaler to request new nodes if the policy allows.

3) Preemption-Aware GPU Sharing *(Details in §5.2)*: This module addresses the tension between resource utilization and responsiveness by implementing cooperative preemption mechanisms. It maintains real-time monitoring of borrowed resources and orchestrates their reclamation when the resource-owning team submits higher-priority workloads.

It directly realizes the **preemption** principle via dynamic, policy-driven rebalancing algorithms capable of rapidly reclaiming GPU resources based on priority signals, urgency, and team policies. This ensures that critical workloads can access the required resources predictably, even under high contention conditions.

4) Resource Scheduling with Hilbert Mapping *(Details in §6)*: This module orchestrates the final placement of tasks by employing spatial locality-aware GPU allocation strategies based on Hilbert space-filling curves [28, 43]. It integrates predicted resource needs and real-time cluster state information to optimize hardware utilization and reduce communication overhead.

The Hilbert mapping scheduler embodies *prioritization* by dispatching tasks according to their priority scores while considering resource fit and hardware locality. Specifically, it maps tasks with high predicted inter-communication needs to GPUs that are physically proximate (e.g., on the same node or connected by a high-speed interconnect), thereby minimizing communication latency and improving the performance of distributed training workloads. This approach ensures higher-priority workloads are allocated resources efficiently in both temporal and spatial dimensions.

4 Resource Allocation Prediction

4.1 Historical Task Data Collection

To establish a reliable task resource prediction model, we conducted systematic data collection of historical task execution. The data is sourced from a production IDP (Intelligent Development Platform) serving 1,000+ concurrent users across 139 physical nodes. The final dataset encompasses 2,500 task samples across diverse workload types, totaling over 8,000 compute hours.

Task Selection. We selected tasks from production environments spanning multiple application domains, including deep learning training, inference, and data preprocessing. To reflect realistic multi-tenant dynamics, the trace specifically captures co-located task interference patterns and the communication overhead of NVLink/InfiniBand for multi-GPU workloads. The tasks utilize frameworks like TensorFlow and PyTorch, with execution times ranging from 30 seconds to 3,600 seconds.

Data Collection. We conducted data collection on the cluster described in Table 1. As quantified in Table 4, our dataset captures diverse multi-tenant patterns: the 160-GPU cluster is dominated by long-running inference (avg. 4 GPUs),

Table 4. Multi-tenant characteristics across the cluster.

GPUs (Size)	Tenants (#)	Users (#)	Shared (Wkly)	Preempt (Wkly)	Tasks (Wkly)
160	5	98	20	22	2,917
400	12	51	3	55	493
1024	3	1054	16	214	8,206

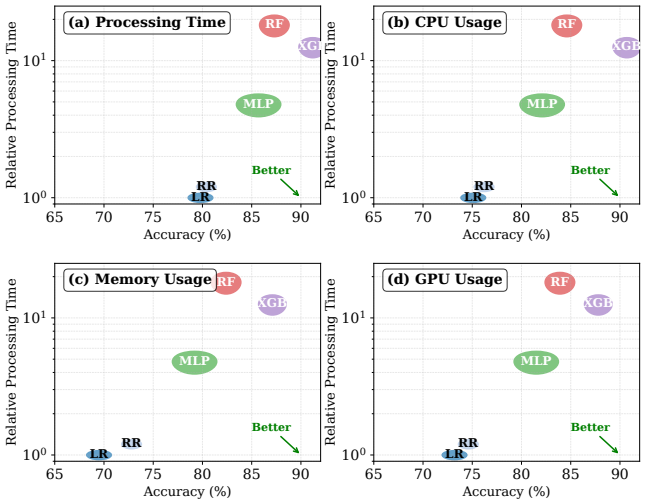


Figure 4. The results of resource prediction using LR, RR, MLP, RF, and XGBoost.

while the 400-GPU cluster exhibits frequent preemptions (55/week) for training experiments. The 1,024-GPU cluster represents extreme diversity, ranging from large-scale 48-GPU training jobs (avg. 52 hours) to massive small-scale bursty tasks (avg. 1.7 GPUs) with high preemption rates. During task execution, the system automatically records resource utilization metrics at 1-second sampling intervals over a 6-month period. Task metadata includes task type, framework, and batch size. The dataset contains three distinct task categories: short tasks (80%, avg. 23 min), medium tasks (10%, avg. 135 min), and long services (10%, avg. 413 min).

Dataset Construction. We preprocessed the collected raw execution data to generate a standardized dataset for model training. Task attributes were encoded as numerical feature vectors, while resource utilization metrics were normalized to the 0-1 range. The final dataset contains 2,500 task records, with each record comprising a 15-dimensional feature vector and a 4-dimensional target vector (execution time, CPU, memory, and GPU utilization).

4.2 Prediction Methods of Wind

Wind uses supervised learning on historical task attributes and resource consumption to predict future requirements. We prioritize lightweight algorithms to meet real-time scheduling demands while maintaining acceptable accuracy, but

also consider more complex models when their accuracy gains justify the overhead.

We evaluate five machine learning algorithms: Linear Regression (LR [52]), Ridge Regression (RR [33]), Multi-Layer Perceptron (MLP [27]), Random Forest (RF [5]), and XGBoost [9], across four prediction targets: processing time, CPU usage, memory usage, and GPU usage. Each algorithm represents different complexity-accuracy trade-offs: LR/RR provide fast linear modeling, MLP captures moderate nonlinearity, while RF/XGBoost handle complex patterns at a higher computational cost. Models are trained dynamically on the 1,000 most recent similar tasks.

Algorithm Selection. Figure 4 shows the accuracy-latency trade-off for each algorithm across prediction targets. We define prediction accuracy as the percentage of predictions with relative error below 20%. The results reveal target-dependent optimal choices: linear models (LR/RR) achieve ~80% accuracy for processing time and CPU usage with minimal latency (sub-0.1 ms), while complex models like XGBoost reach the highest accuracies (up to 91.2% for processing time and 90.7% for CPU usage). Although XGBoost is roughly 12.6× slower than LR, its absolute inference time remains on the order of 1–2 ms, which is well within the acceptable range for online scheduling.

Based on these trade-offs, Wind adopts XGBoost as the primary prediction model, leveraging its consistently superior accuracy across all four targets (processing time, CPU, memory, GPU). To further optimize latency-sensitive scenarios, lightweight models such as LR/RR or MLP are employed as fallbacks—e.g., for rapid estimation in ultra-tight scheduling loops or on resource-constrained devices. This strategy ensures high prediction fidelity (above 87% across all metrics with XGBoost) while preserving sub-millisecond responsiveness when necessary, making it well-suited for real-time scheduling in dynamic environments.

4.3 Prediction Model Maintenance and Robustness

To ensure the long-term fidelity of resource forecasts in dynamic cluster environments, Wind implements a systematic model maintenance and error-handling pipeline.

Model Retraining and Feature Importance. The system adopts a "daily-ingestion, weekly-update" strategy. While production data is accumulated in real-time to capture potential shifts in workload patterns, we perform full model retraining on a weekly basis. This cadence is justified by our observation that in typical multi-tenant AI clusters, the tenant base and their core model architectures (e.g., recurring LLM pre-training or fine-tuning pipelines) remain relatively stable over several months.

Handling Prediction Errors and Safety Margins. Despite a baseline accuracy of 87%, mispredictions are inevitable. Wind handles these via a conservative resource fallback mechanism. When the prediction model reports high variance (low confidence) or encounters out-of-distribution task attributes, the system applies a "safety margin" by reverting to the user-requested resource values as the upper bound for scheduling.Crucially, the impact of such mispredictions is mitigated by two architectural features: *1) Hilbert Mapping Resilience:* Since Hilbert curves optimize for spatial locality (placing related tasks on proximate nodes), a slight error in execution time estimation primarily affects the temporal packing efficiency rather than physical interconnect contention. *2) K8s Enforcement:* Wind acts as a high-level orchestrator that modifies the task's resource specifications. Kubernetes then strictly enforces these modified limits at the container level, ensuring that even under-predicted tasks do not cause resource starvation for co-located neighbors.

5 Fine-grained Resource Management

Fine-grained resource management enhances resource efficiency and handles multi-priority tasks by enabling resource preemption and isolation mechanisms. Wind aims to leverage idle resources during off-peak times and gracefully degrade in the face of increased system pressure, avoiding over-occupation of system resources and thus improving overall resource scheduling efficiency.

5.1 Dynamic Resource Quota Management

The soft preemption mechanism achieves flexible resource orchestration through dynamic management of heterogeneous resource quotas, encompassing CPU, memory, and GPU resources. For `Burstable` Pods, the system enables the configuration of variable resource bounds to adapt resource consumption according to real-time system load conditions. Specifically, each `Burstable` Pod is provisioned with adjustable CPU, memory, and GPU quotas that define both baseline requirements and peak allocation limits. When requesting resources, the Pod declares the minimum resource guarantee (e.g., `2 CPUs, 4Gi memory, and 1 GPU`) alongside the maximum resource ceiling (e.g., `8 CPUs, 16Gi memory, and 4 GPUs`). This elastic resource model ensures that Pods can exploit surplus resources during low-contention periods while gracefully degrading resource usage under high system load, thereby preventing resource starvation and maintaining system stability.

The resource management orchestration leverages Kubernetes' `kubelet` as the primary enforcement agent for multi-resource coordination. For CPU resources, the system employs dynamic `cpu.shares` adjustment to modulate the CPU time allocation for `Burstable` Pods. Under high node utilization, the scheduler reduces CPU shares (e.g., from 1024 to 512) to prioritize critical workloads while maintaining fairness guarantees. Memory management operates through continuous monitoring of `memory.pressure` metrics, where the system progressively decreases `memory.high` thresholds for `Burstable` Pods when memory contention exceeds predefined bounds, triggering controlled memory reclamation

to prevent out-of-memory conditions. For GPU resources, the system utilizes device-plugin interfaces to implement fractional GPU sharing and dynamic GPU memory allocation. The scheduler monitors GPU utilization metrics and memory fragmentation patterns to redistribute GPU resources among competing Pods, ensuring optimal GPU memory locality and minimizing inter-device communication overhead in multi-GPU scenarios.

5.2 Preemption-Aware GPU Sharing

For dynamic adjustment of GPU resources, the system introduces a preemption-aware GPU sharing mechanism to further enhance GPU resource utilization. By extending the device plugin, the system allows Pods to declare their GPU time slice requirements, enabling the system to set preemption policies for GPU resources and define the duration of each time slice. For example, a Pod can declare its need for preemptible GPU resources and specify that it will occupy 200 milliseconds of GPU time per second. This allows for more refined GPU resource allocation, particularly in multi-task scenarios where multiple tasks share a single GPU, thus avoiding resource wastage.

In terms of preemption strategy, the system provides both active relinquishment and passive preemption. To achieve millisecond-level passive preemption, we modify the CUDA scheduler within the NVIDIA open-source kernel driver. This implementation enables a hardware-level pause-resume mechanism: when a high-priority task arrives, the driver triggers a context switch that saves current register states and execution metadata directly into the GPU's High Bandwidth Memory (HBM), rather than offloading to host memory. By minimizing host-to-device synchronization, the system reduces the total context switch latency to approximately 5-10 ms. The active relinquishment strategy complements this by requiring tasks to periodically check their remaining time slice and proactively save model checkpoints before the time slice is exhausted. In contrast, the passive preemption strategy leverages the kernel-level scheduler to suspend kernels or sends a `SIGTERM` signal to the task via the device plugin when the task does not release GPU resources in time. After a specified grace period (defined by `terminationGracePeriodSeconds`), the task is forcibly terminated to ensure that higher-priority tasks have sufficient access to the GPU resources.

5.3 Isolation Guarantee for Exclusive Resources

In Wind, the resource preemption mechanism serves as the fundamental basis for ensuring service quality for AI tasks with different priorities. It primarily encompasses strong isolation and fine-grained scheduling for CPU, memory, and GPU resources.

1) CPU/Memory Isolation

The system employs the `cgroups v2` mechanism in the Linux kernel to isolate and control different QoS levels for Pods. For high-priority tasks marked as 'Guaranteed', the system allocates a fixed real-time execution window by configuring the `cpu.rt_runtime_us` parameter and combines it with the `SCHED_FIFO` real-time scheduling policy. This ensures that such Pods have scheduling priority over low-priority processes running under regular scheduling policies, such as CFS (Completely Fair Scheduler [38]). However, as Kubernetes does not enable real-time scheduling by default, the system requires pre-configuration of the nodes and runtime extension to explicitly enable real-time privileges for these Pods.

For memory isolation, the system uses the `memory.high` parameter to set memory usage limits for non-critical tasks (e.g., `BestEffort` Pods). When the node enters a memory pressure state, the system gradually compresses or terminates non-critical tasks to avoid excessive memory consumption, ensuring that critical tasks maintain available memory.

2) GPU Exclusive Allocation

In GPU resource isolation, the system employs both spatial and temporal isolation mechanisms to support exclusive access and preemption capabilities for high-priority tasks. Our spatial isolation follows a two-tier approach: we perform static rebalancing during off-peak windows based on 7-day usage patterns to reduce overhead, while utilizing HAMI for dynamic sharing to enable flexible allocation without disruptive hardware resets. For temporal isolation, the system customizes the NVIDIA driver to introduce a **Preemptive Time-Slicing** mechanism. This mechanism enables time-slice scheduling for the CUDA kernel, and when a task exceeds its allocated time slice, the kernel scheduler forcibly interrupts the current GPU execution, making room for higher-priority tasks. This mechanism overcomes the limitations of traditional GPU scheduling, where tasks cannot be preempted, thus significantly improving preemption response time and task-switching granularity.

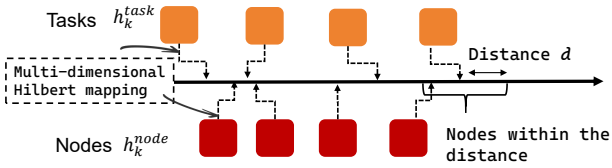


Figure 5. Hilbert mapping in Wind: Tasks and nodes are ordered using multi-dimensional Hilbert mapping, where nodes within the Hilbert distance threshold of a given task are prioritized for task placement.

6 Hilbert-based Multi-dimensional Resource Scheduling Algorithm

To address the challenge of matching multi-attribute tasks with heterogeneous cluster resources, we designed and implemented a novel scheduling system. Its core is a scheduling algorithm that leverages Hilbert space-filling curves to map high-dimensional resource vectors onto a one-dimensional

space. This mapping transforms the complex multi-dimensional best-fit problem into an efficient one-dimensional proximity search, enabling fine-grained, priority-aware task placement.

6.1 The framework

The scheduling framework is composed of three key components that work in concert: a centralized **Scheduler Core**, a per-node **Node Agent**, and a shared **State Manager**.

Scheduler Core: This is the central decision-making entity. It is designed to be stateless to ensure scalability and fault tolerance. It continuously monitors the task queues and node states stored in the State Manager and executes a three-phase scheduling pipeline (§6.3) to make placement and preemption decisions.

Node Agent:A daemon process running on each worker node. Its responsibilities are twofold: (1) monitoring local resource utilization (CPU, memory, GPU), running task statuses, and real-time load, and periodically reporting these as 6-dimensional state vectors to the State Manager; (2) executing commands from the Scheduler Core, such as launching a container, terminating a task, or signaling a process for resource reclamation (e.g., via `SIGUSR1`).

State Manager: A logically centralized, highly available key-value store. It maintains the global state of the cluster, including: (1) multiple priority-based task queues, (2) the Hilbert value and resource state for every node, and (3) metadata for ongoing preemption operations. This decoupled framework allows the Scheduler and Agents to operate asynchronously.

6.2 The model

The foundation of our scheduler is a compact multi-dimensional model for both tasks and nodes, capturing essential resource demands together with high quality services. To avoid the instability of high-dimensional Hilbert mappings, we separate hard resource feasibility from QoS-sensitive placement, resulting in a more efficient 3–4D representation.

Definition. A task j_k is described by $j_k = (r_k, q_k, \xi_k, \tau_k)$, where r_k is the aggregated demand for CPU, memory, and GPU (computed as a weighted sum of normalized resources), q_k is its priority level, ξ_k indicates preemptibility, and τ_k is the estimated duration. A node n_i is formally defined as a tuple $n_i = (r_i, \bar{q}_i, \bar{\xi}_i, \bar{\ell}_i)$, where $r_i = (\text{cpu}_i, \text{mem}_i, \text{gpu}_i)$ represents the aggregated available resource capacity in terms of CPU cores, memory, and GPU count, respectively. The parameter $\bar{q}_i$ summarizes the average priority of the tasks currently executing on the node, while $\bar{\xi}_i$ denotes the ratio of preemptible tasks, incorporating per-GPU tracking to facilitate fine-grained scheduling. Finally, $\bar{\ell}_i$ serves as a normalized load metric, where higher values indicate heavier node utilization, a characteristic preferred during Hilbert-based matching to promote workload consolidation.

Hilbert Mapping. Resource feasibility ($r_i \geq r_k$) is checked first to ensure hard constraints are met. For QoS-sensitive scheduling, we employ a mapping function $H : \mathbb{R}^d \rightarrow \mathbb{N}$ ($d = 3$ or 4 depending on configuration) that projects task and node vectors into a one-dimensional Hilbert value. To enforce priority, the system maintains P distinct mappings ${H^{(p)}}_{p=1}^{P}$, where higher-priority mappings weigh QoS attributes more strongly than load. The Hilbert value for a task j_k with priority q_k is $h_k^{task} = H^{(q_k)}(j_k)$, computed by the Scheduler Core upon task arrival. A node's Hilbert value, h_i^{node}, is updated dynamically by its Node Agent.

The effectiveness of the Hilbert-based mapping stems from the locality-preserving properties inherent in Space-Filling Curves[43]. By projecting multi-dimensional resource vectors into a one-dimensional continuum while maintaining spatial proximity, the Hilbert curve naturally clusters similar resource footprints. In this framework, the input vector r_k can be derived either from dynamic usage predictions or, in the absence of such information, from the static resource declarations provided in the task specification. While accurate predictions can further refine the alignment by reflecting actual runtime behavior, the fundamental advantage of the Hilbert approach, reducing multi-dimensional fragmentation through geometric clustering, remains robust. This ensures that the scheduler efficiently addresses the "resource matching" problem by optimizing bin-packing density, a spatial benefit that persists even when temporal predictions are unavailable or imprecise.

Distance-based Filtering. Matching is performed by comparing distances in the Hilbert space:

$$d(j_k, n_i) = |h_k^{task} - h_i^{node}|$$

A smaller distance indicates a better fit in terms of QoS and workload state. For a task j_k with priority q_k, the candidate set is defined as

$$C_k = \{n_i \in \mathcal{N} \mid r_i \geq r_k \wedge d(j_k, n_i) \leq \theta^{(q_k)}\}$$

where $\theta^{(q_k)}$ is a priority-dependent distance threshold. High-priority tasks use a small θ, enforcing a strict match, while low-priority tasks are allowed a larger threshold, increasing their placement chances.

6.3 The pipeline

As shown in the Figure 6, the Scheduler Core executes the following pipeline for each scheduling cycle, operating on the head of the highest-priority non-empty task queue.

Phase 1: Candidate Filtering and Adaptive Scanning. For a task j_k, the scheduler first determines its Hilbert anchor h_k^{task} in $O(1)$ time and queries the State Manager to retrieve a list of nodes that satisfy the basic resource requirements. To accelerate subsequent matching and positioning, node states in the State Manager are indexed by their Hilbert values using a B+-tree. The scheduler performs a range query on this index for candidate nodes with $h_i^{node} \in [h_k^{task} - \theta^{(p_k)}, h_k^{task} + \theta^{(p_k)}]$. The intersection of these two sets

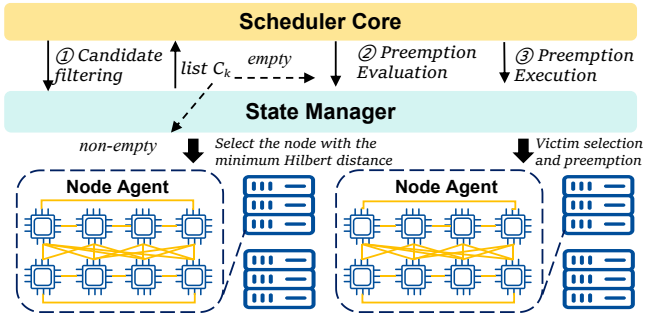


Figure 6. The pipeline of the Hilbert-based multi-dimensional resource scheduling algorithm is integrated into a framework composed of the Scheduler Core, Node Agent, and State Manager.

forms the final candidate list C_k. Starting from this anchor, the scheduler performs a second-round linear scan ($O(n)$) on the candidate set C_k to verify resource feasibility ($r_i \geq r_k$) and optimize load balance. This design ensures structural robustness: even with low prediction accuracy, the Hilbert curve maintains relative rankings. If C_k is non-empty, the node with the minimum Hilbert distance is chosen, followed by sending a `LaunchTask` command to the corresponding Node Agent, and the task is dequeued. Otherwise, the scheduler proceeds to Phase 2.

Phase 2: Preemption Evaluation. If no suitable node is found, the scheduler evaluates the viability of preemption. A preemption is considered beneficial if there exists a node n_i that, after preempting a set of lower-priority tasks $\mathcal{P}_i$, could accommodate the high-priority task j_k. To prevent system thrashing, the scheduler enforces a cooldown mechanism: a preemption on node n_i is suppressed if the node has undergone preemption within a dynamically calculated window $T_{cooldown}$, which is proportional to the recent preemption frequency on that node.

Phase 3: Adaptive Preemption Execution. If preemption is deemed necessary and viable, the scheduler selects the best preemption target.

Victim Selection: The scheduler scores potential victim nodes using a cost function $S(n_i) = \gamma \cdot d(j_k, n_i) + C(n_i)$, which minimizes both the Hilbert distance (for a better fit) and the preemption cost $C(n_i)$. The cost $C(n_i)$ penalizes preempting tasks that have run for a long time or have high (relative to other preemptible tasks) priority, thereby preserving work.

Execution and State Reconciliation: Once the victim node n_i and victim tasks $\mathcal{P}_i$ are identified, the scheduler sends a `PreemptTask` command to the Node Agent on n_i. For GPU tasks, our Node Agent implements a two-stage preemption: it first sends `SIGUSR1` to the task's process group, allowing for voluntary checkpointing and resource release. If the resources are not freed within a short timeout, `SIGTERM` is sent for forced termination. The preempted tasks are requeued in the State Manager, and the released resources trigger an immediate rescheduling attempt for task j_k. This closed-loop process ensures that resources are rapidly re-allocated to high-priority workloads.

Complexity and Scalability Analysis. The scheduling pipeline balances constant-time indexing with linear adaptive scanning. Specifically, the Hilbert anchor positioning achieves $O(1)$ complexity, while the adaptive scan follows a worst-case $O(n)$ complexity. Overall, the Hilbert scheduling computation is completed on a 100-nanosecond scale, even at a scale of 1,024 nodes.

7 Evaluations

7.1 Evaluation Configurations

Experimental Setup: Our experiments are conducted on a 139-node physical cluster composed of five different hardware configurations, as detailed in Table 1. All nodes are interconnected via a 100GbE RoCE v2 (RDMA) network. This high-performance interconnect is crucial for modern distributed ML workloads and ensures that the network does not become a bottleneck, allowing us to accurately assess the scheduler's performance itself. The software stack across all nodes includes *Ubuntu 22.04*, *Kubernetes v1.28*, and *NVIDIA Driver 535+*. To eliminate network latency during job startup, all necessary container images are pre-pulled to the nodes.

Benchmark Comparison: We compare the performance of Wind against four representative baseline schedulers: *(1) Kubernetes Default:* The default Kubernetes scheduler, which serves as the de facto industry standard baseline. *(2) DRF (Dominant Resource Fairness [16]):* A classic algorithm designed to provide fairness across users with multi-resource demands. *(3) Synergy [35]:* A state-of-the-art scheduler that is a direct competitor in multi-dimensional resource-aware scheduling.

All the schedulers operate under identical experimental conditions and workloads, with a unified configuration across all test dimensions.

Workloads: To ensure our evaluation reflects real-world scenarios, we use a workload derived from production traces of our ML platform. We have categorized the tasks from the trace into three distinct workload scenarios to stress-test different aspects of the schedulers: *(1) Fixed tasks:* Periodic, fixed-interval compute tasks with stable and predictable resource footprints. Their arrival patterns and runtimes show low variance. *(2) Burst Tasks:* High-priority, time-critical workloads requiring immediate resource allocation, such as inference traffic spikes and emergency parameter tuning. These bursts are typically short-lived but intense. *(3) Hybrid Workload:* Mixed workloads combining periodic background tasks with burst interactive requests, simulating production AI platforms with concurrent training and serving demands.

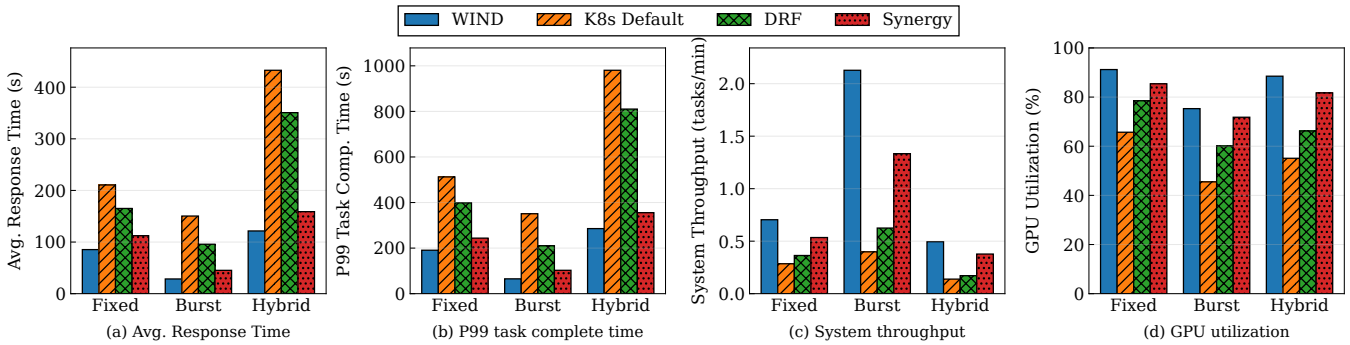


Figure 7. Performance evaluation (average response time, P99 task complete time, system throughput, GPU utilization) under different task submission intervals.

7.2 The Basic Performance

We evaluate the performance of Wind across three workloads: batch training, model inference, and hybrid load, comparing it with `K8s Default`, `DRF`, and `Synergy`. The results are summarized in Figure 7, which presents average response time, P99 job completion time (JCT), system throughput, and GPU utilization.

In the batch training workload, Wind achieves the highest System Throughput (1.42 min/job) and GPU Utilization (91.2%), significantly outperforming all baselines. This is due to its accurate resource demand prediction and multi-dimensional resource packing based on Hilbert curves, which reduces resource fragmentation. In contrast, `K8s Default` exhibits low throughput and poor resource utilization due to the lack of resource awareness. For model inference, Wind delivers the lowest Average Response Time (28.5s) and P99 Completion Time (65.1s), outperforming `Synergy` by 37% in response time. This is achieved through its ability to quickly match small, latency-sensitive jobs with fragmented resources. `K8s Default` fails to prioritize latency-sensitive tasks, resulting in significantly higher response times. In the hybrid load scenario, Wind maintains the lowest Average Response Time(121.5s), 28% better than `K8s Default`, by leveraging intelligent preemption. It preempts lower-priority jobs to meet the QoS of high-priority inference tasks. Without effective preemption, `K8s Default` and `FIFO` suffer from severe delays, with P99 JCT exceeding 1500 seconds.

Across all three workloads, Wind outperforms all baselines by efficiently balancing throughput, latency, and resource utilization. Its combination of resource prediction, multi-dimensional mapping, and preemption offers an effective scheduling solution for large-scale GPU clusters.

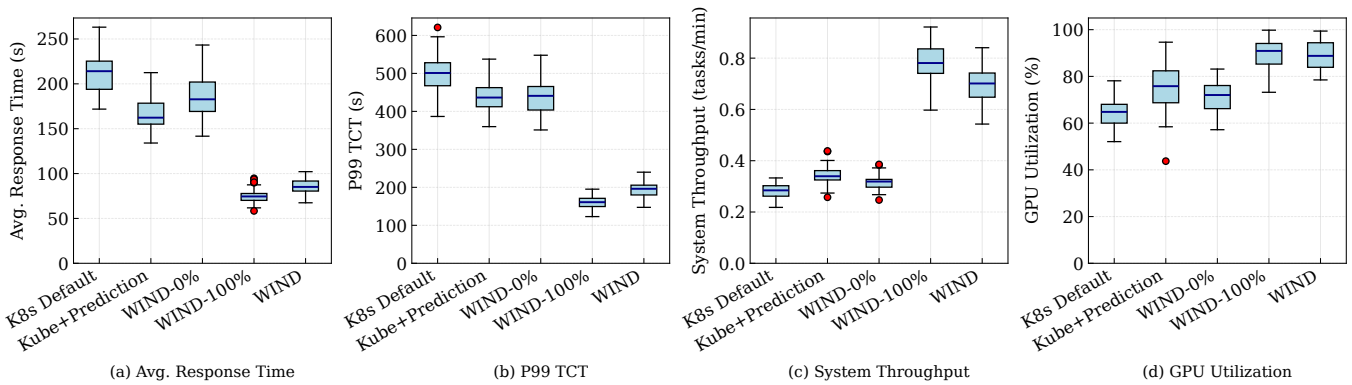
7.3 Ablation Study: Prediction and Hilbert Scheduling

To decompose the performance gains of `WIND`, we conducted an ablation study focusing on two core components: the Prediction Module and the Hilbert-based Multidimensional Packing. We evaluate five configurations: (1) `K8s Default`: Baseline without prediction or Hilbert optimization; (2) `Kube +Prediction`: Integrates prediction into the default scheduler (demonstrating the "Prediction-only" gain); (3) `WIND-0%`: Employs Hilbert scheduling but with zero-accuracy/random predictions (demonstrating the "Hilbert-only" gain); (4) `WIND-100%`: The theoretical upper bound with Hilbert and perfect predictions; (5) `WIND`: The full system with realistic prediction and Hilbert scheduling. Figure 8 presents the boxplots for these configurations.

Contribution of Prediction Module. Comparing `K8s Default` with `Kube+Prediction` highlights the benefit of foresight alone. As shown in Figure 8(a-c), adding prediction to the standard K8s scheduler leads to a downward shift in latency and an upward shift in throughput. Quantitatively, the median response time improves by 21%, and the system throughput increases from 0.28 to 0.35 tasks/min (a 25% improvement). Figure 8(d) shows a tighter GPU utilization distribution (median increased from 65.7% to 74.3%), proving that even without Hilbert optimization, prediction reduces resource stragglers.

Robustness via Hilbert Scheduling. A key finding is the performance of `WIND-0%`. Even when the prediction accuracy is forced to zero, the Hilbert-based packing strategy significantly outperforms `K8s Default`. The median response time drops from 210.6s to 180.3s, and throughput increases to 0.32 tasks/min. Crucially, `WIND-0%` exhibits much lower variance (shorter whiskers in the boxplots) than `K8s Default`. This demonstrates that the Hilbert curve mapping provides a robust structural foundation by naturally reducing resource fragmentation, regardless of prediction quality.

Synergy and Upper Bound. The full `WIND` system achieves its best performance by coupling both modules. While `WIND-100%` sets the theoretical ceiling (e.g., 0.78 tasks/min throughput, 94.1% GPU utilization), the standard `WIND` with its real-world model operates remarkably close to this optimum. It achieves a throughput of 0.70 tasks/min and 91.2% GPU utilization. The gap between `WIND-0%` and `WIND` (0.32 vs 0.70 tasks/min) quantifies the additional value that accurate temporal predictions bring to the spatial packing process.



(a) Avg. Response Time (b) P99 TCT (c) System Throughput (d) GPU Utilization

Figure 8. Performance of ablation study: Kube+Prediction (Prediction-only) vs. WIND-0% (Hilbert-only).

These results validate the functional independence and combined efficacy of our design: (1) Prediction provides the necessary look-ahead to avoid future bottlenecks; (2) Hilbert Scheduling ensures stable and efficient spatial packing even under prediction uncertainty; and (3) together, they bridge the gap between reactive baselines and ideal proactive scheduling.

7.4 Performance on Task Submitted Patterns

We constructed a comprehensive workload comprising 200 tasks categorized into eight distinct resource consumption profiles, ranging from 1/8 to full resource (8/8) utilization. These profiles correspond to CPU allocations from 2 to 16 cores and memory requirements from 8GB to 64GB. Task design follows the principle of positive correlation between resource demands and execution time, where higher resource-consuming tasks exhibit proportionally longer runtimes, thereby simulating the characteristics of real-world compute-intensive applications.

To evaluate scheduler performance under varying system pressures, we established five task submission patterns: zero-interval submission (200 tasks submitted simultaneously, simulating burst peak loads), 10-second intervals (high-load scenarios), 20-second intervals (medium load), 40-second intervals (light load), and 80-second intervals (system idle state). This progressive load design enables systematic observation of scheduling strategy behaviors across system states ranging from extreme bursts to relative quiescence.

Figure 9 presents a comparative performance analysis of Wind against three baseline schedulers: K8s Default, DRF, and Synergy. The results across five distinct task submission patterns reveal Wind's substantial advantages in response time, system throughput, and resource utilization metrics.

Wind consistently outperforms all baseline schedulers, achieving significant response time reductions. For instance, compared to the K8s Default scheduler, it registers improvements of 33-48% across all load patterns. Under burst load scenarios (0-second intervals), Wind dramatically reduces response time from 4.2 seconds to 2.8 seconds, while maintaining sub-second response (0.5 seconds) during idle periods. This consistent improvement stems from Wind's resource quota management established through historical load analysis, enabling proactive resource window identification prior to task arrival. The Hilbert curve optimization compresses candidate node localization to under 100 milliseconds, achieving order-of-magnitude efficiency gains compared to the linear scanning strategies of other schedulers.

System throughput improvements are most pronounced under high-pressure scenarios, where Wind demonstrates clear superiority. In the burst load scenario, it achieves a throughput of 8.5 tasks/min, a 46.6% gain over the K8s Default scheduler (5.8 tasks/min) and also significantly surpassing DRF and Synergy. The technical foundation of this enhancement lies in Wind's transformation of traditional multi-dimensional resource matching into one-dimensional proximity search. Spatial locality preservation ensures natural clustering of similar resource requirements, thereby maintaining high task completion rates even under intensive resource contention.

Wind maintains consistently high resource utilization, staying above 92% across all patterns. This contrasts sharply with the wider fluctuation and lower efficiency of baseline schedulers, such as K8s Default (78.4-92.7%). The 17.7% improvement in burst scenarios (92.3% vs. 78.4%) particularly highlights Wind's **dynamic resource reclamation capability**. When compute nodes complete tasks and release resources, Wind utilizes real-time Hilbert values to immediately identify these resource opportunity windows and achieve instant reallocation, reducing resource reuse intervals from traditional minute-scale to second-scale operations.

7.5 Resource Saving

Figure 10 further validates the advantages of the Wind scheduler from the perspective of resource savings in a 20-node cluster environment. In the resource usage comparison experiment, Wind demonstrates superior resource consolidation

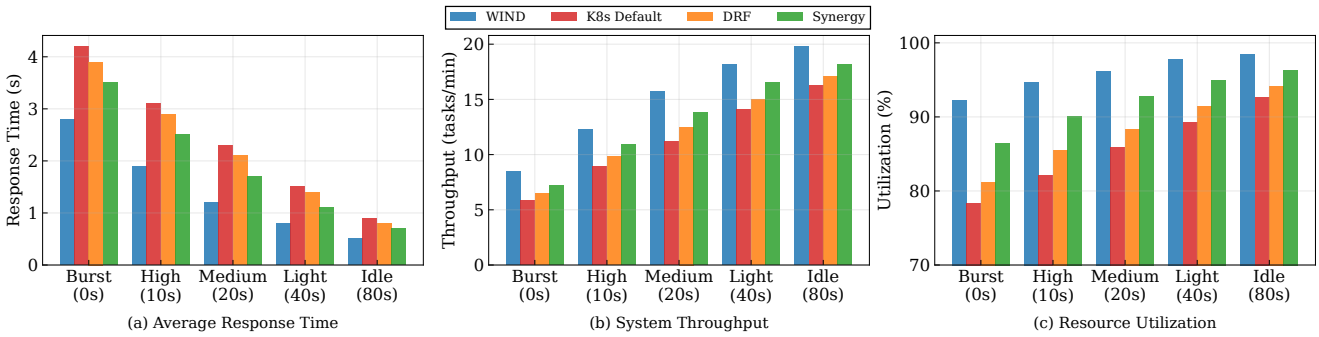


Figure 9. Performance evaluation (average response time, system throughput, resource utilization) under different task submission intervals.

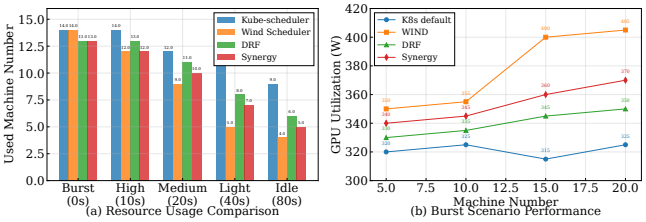


Figure 10. Performance of resource saving.

Table 5. Reliability of Wind.

Failure Category	Count	%	Rate/1K days
Faulty GPU	148	30.1	0.17
GPU HBM3 Memory	72	17.2	0.08
GPU SRAM Memory	19	4.5	0.02
GPU System Processor	17	4.1	0.02
Silent Data Corruption	6	1.4	0.01
GPU Thermal Issues	6	1.4	0.01
Total	**268**	**58.7**	**0.31**

capabilities across all workload intensities. Under medium-load scenarios, Wind requires only 9 machines to complete tasks, representing a 25% reduction in resource consumption compared to K8s Default's 12 machines. The performance gap becomes even more pronounced in lighter workloads: in light-load and idle scenarios, Wind uses 5 and 4 machines respectively, compared to Kube-scheduler's 11 and 9 machines, demonstrating significant resource-saving effects. This differentiated performance verifies the technical advantages of advanced scheduling algorithms in resource saving.

GPU utilization analysis in burst scenarios reveals the performance potential of the Wind scheduler across different cluster scales (5 to 20 machines). Wind's GPU utilization gradually increases from an initial 350W at 5 machines to over 405W at 20 machines, achieving a 20-25% performance improvement compared to K8s Default's stable fluctuation range of 315-325W. DRF and Synergy demonstrate intermediate performance levels, with GPU utilization reaching 350W and 370W respectively at 20 machines. This gap holds important cost-efficiency value for resource-intensive AI workloads, fully proving the effectiveness of the Wind scheduler's predictive mechanism combined with geometric optimization in large-scale cluster environments.

7.6 Reliability

We systematically evaluate Wind's reliability in a large-scale heterogeneous cluster of 13,302 nodes, comprising over 100,000 NVIDIA GPUs ranging from H100/H800 to the RTX 4090 series. Each node is equipped with up to 192 CPU cores and 2 TB RAM, utilizing NVLink for intra-node and IB/RoCE for inter-node communication with dedicated 4-lane bonded storage networking. Reliability is enhanced through hierarchical monitoring (ranging from 100 ms to 15 s) and a three-tier storage hierarchy consisting of Local NVMe, Distributed Cache, and GPFS for efficient checkpointing. Our evaluation spans diverse training workloads across the entire cluster.

During 54 days of continuous monitoring, we collected comprehensive operational data and recorded 419 system interruption events. Analysis reveals that GPU-related failures remain the dominant interruption source, accounting for 63.9% (268 incidents) of total events, with detailed failure types shown in Table 5. Among GPU failure categories, general GPU functional failures constitute 30.1% and HBM3 high-bandwidth memory failures represent 17.2%. In deployment, Wind includes an automated self-healing workflow for GPU faults. When a failure is detected, the system marks the node unschedulable, waits for running tasks to finish, and then replaces the node. This reduces manual work and shortens average recovery time from about 12 hours to roughly 2 hours.

Experimental results demonstrate that Wind achieves two significant reliability improvements. First, by integrating history-based task parameter prediction, the system accurately estimates resource requirements based on historical execution patterns, reducing task failures and system instability caused by resource misconfiguration. Second, Wind's fine-grained scheduling mechanisms combined with dynamic resource quotas and preemption-aware policies substantially enhance fault recovery capabilities. Measurements show that

over 90% of GPU-related failures can be rapidly resolved through automated restart procedures or resource reallocation strategies without manual intervention or hardware replacement.

8 Related Works

AI Workload Scheduling and Resource Management. Scheduling for AI workloads often relies on established techniques like resource partitioning and queuing (e.g., Tiresias [17], ASTRAEA [64]), workload-resource affinity modeling [14, 20, 53], and flexible job arrangements via time slicing or migration [8, 23, 57]. While systems like Kubernetes [7] provide basic quotas, and others explore time-sharing [1, 57], they often lack adaptive allocation for dynamic workloads.

Dynamic Colocation and GPU Sharing. Mechanisms for dynamic GPU colocation and sharing frequently utilize spatial or spatio-temporal multiplexing (GSLICE [13], Gpulet [10]) and interference management [62]. Other works improve throughput via model placement and parallelism [22, 31]. However, these approaches are often tailored for inference workloads and lack a unified framework integrating prediction and preemption-aware scheduling.

Predictive and Locality-Aware Allocation. Predictive scheduling has been explored using machine learning for job completion prediction (Prophet [67]), reinforcement learning for allocation (Morpheus [12]), and workload forecasting for serverless computing [18]. In parallel, locality-aware allocation is addressed through basic NUMA-aware mechanisms [7] or affinity-based placement [58]. Although space-filling curves are used for data placement in distributed systems [28, 43], their application to priority-aware GPU scheduling remains largely unexplored, highlighting a key research gap.

9 Discussion and Lessons Learned

Stability and Simplicity in Prediction. Real-world deployment reveals that prediction stability is as critical as accuracy. To handle distribution shifts in production, Wind employs conservative fallback mechanisms and hybrid policies. The robustness of Hilbert-based spatial packing is vital here, preserving locality and load balance even when predictions are imperfect. Furthermore, we found that simplicity often outweighs marginal optimality gains. By projecting multi-dimensional resources into a one-dimensional Hilbert space, Wind ensures deterministic placement and reduces scheduling jitter, particularly for latency-sensitive workloads.

Balancing Isolation, Sharing, and Preemption. A fundamental tension exists between resource isolation and utilization. While naive sharing can cause 18–29% performance degradation, strict isolation leads to underutilization. Wind resolves this through predictive scheduling, elastic quotas, and preemption-aware sharing. However, preemption must be used sparingly due to checkpointing and cache overheads. We implement cooldown windows and cost modeling to ensure preemption remains policy-driven and low-frequency, preserving overall system throughput and stability.

Fairness, Efficiency, and Future Directions. Traditional resource allocation models emphasize tenant isolation but often cause resource fragmentation and reduced throughput in high-density GPU clusters. Wind replaces rigid partitioning with priority-aware Hilbert mappings and flexible quota governance, enabling various AI workloads to co-locate with minimal interference. Our operational experience shows that hierarchical prioritization is not optional but a requirement for handling volatility in multi-tenant environments.

Building on this foundation, we plan to extend Hilbert-based scheduling to explicitly encode network topology and interconnect affinity. By mapping communication patterns, for example NVLink and InfiniBand constraints, into the space-filling curve, the scheduler can naturally mitigate cross-node bottlenecks in large-scale distributed training. Moving from reactive to proactive orchestration remains a key objective. Integrating lightweight online learning into the scheduling loop would allow Wind to anticipate workload phase shifts and adjust placement before contention emerges. Our goal is to evolve GPU management into a self-optimizing closed-loop system that uses real-time telemetry, including compute performance and hardware health, to recalibrate policies dynamically. This direction positions Wind as a core component of next-generation autonomous infrastructure that can handle increasing heterogeneity in both hardware and workloads.

10 Conclusion

We present Wind, a novel scheduling framework designed to overcome critical challenges in modern AI cluster resource management, including inefficient multi-dimensional resource coordination, limited predictive capabilities, and the inherent tension between isolation and sharing. Wind achieves this through a unique combination of predictive modeling and geometric resource mapping. By replacing reactive scheduling with proactive resource planning via history-based parameter prediction and implementing Hilbert curve-based multi-dimensional resource allocation, Wind fundamentally transforms how heterogeneous computing resources are managed in production AI environments.

Our evaluation shows Wind outperforms existing solutions across key metrics, particularly excelling with bursty workloads that typically challenge conventional schedulers. The system has been successfully deployed in production as part of the `Baihai` IDP platform, where it has delivered efficient and reliable computing services to thousands of clients for multiple years.

Acknowledgments

We appreciate our shepherd Rohan Sanjeev Patil and anonymous reviewers' constructive comments for improving the quality of this paper.

References

[1] Sohaib Ahmad, Hui Guan, Brian D Friedman, Thomas Williams, Ramesh K Sitaraman, and Thomas Woo. 2024. Proteus: A high-throughput inference-serving system with accuracy scaling. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 318–334.

[2] Xin Ai, Zijian Li, Yuanyi Zhu, Zixuan Chen, Sen Liu, and Yang Xu. 2025. NetJIT: Bridging the Gap from Traffic Prediction to Preknowledge for Distributed Machine Learning. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 1 (2025), 1–25.

[3] Hadeel Albahar, Shruti Dongare, Yanlin Du, Nannan Zhao, Arnab K Paul, and Ali R Butt. 2022. Schedtune: A heterogeneity-aware gpu scheduler for deep learning. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 695–705.

[4] Laith Alzubaidi, Jinglan Zhang, Amjad J Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, José Santamaría, Mohammed A Fadhel, Muthana Al-Amidie, and Laith Farhan. 2021. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of big Data* 8 (2021), 1–74.

[5] Gérard Biau and Erwan Scornet. 2016. A random forest guided tour. *Test* 25, 2 (2016), 197–227.

[6] Arthur R Butz. 2006. Alternative algorithm for Hilbert's space-filling curve. *IEEE Trans. Comput.* 100, 4 (2006), 424–426.

[7] Carmen Carrión. 2022. Kubernetes scheduling: Taxonomy, ongoing issues and challenges. *Comput. Surveys* 55, 7 (2022), 1–37.

[8] Shubham Chaudhary, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, and Srinidhi Viswanatha. 2020. Balancing efficiency and fairness in heterogeneous GPU clusters for deep learning. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.

[9] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.

[10] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on Multi-GPU servers with Spatio-Temporal sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 199–216.

[11] Arnab Choudhury, Yang Wang, Tuomas Pelkonen, Kutta Srinivasan, Abha Jain, Shenghao Lin, Delia David, Siavash Soleimanifard, Michael Chen, Abhishek Yadav, et al. 2024. MAST: Global scheduling of ML training across Geo-Distributed datacenters at hyperscale. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 563–580.

[12] Sina Darabi, Mohammad Sadrosadati, Negar Akbarzadeh, Joël Lindegger, Mohammad Hosseini, Jisung Park, Juan Gómez-Luna, Onur Mutlu, and Hamid Sarbazi-Azad. 2022. Morpheus: Extending the last level cache capacity in GPU systems using idle GPU core resources. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 228–244.

[13] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. 2020. Gslice: controlled spatial sharing of gpus for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. 492–506.

[14] Wei Gao, Zhisheng Ye, Peng Sun, Yonggang Wen, and Tianwei Zhang. 2021. Chronus: A novel deadline-aware scheduler for deep learning training jobs. In *Proceedings of the ACM Symposium on Cloud Computing*. 609–623.

[15] Yanjie Gao, Yichen He, Xinze Li, Bo Zhao, Haoxiang Lin, Yoyo Liang, Jing Zhong, Hongyu Zhang, Jingzhou Wang, Yonghua Zeng, et al. 2024. An empirical study on low gpu utilization of deep learning jobs. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[16] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*.

[17] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 485–500.

[18] Luanzheng Guo, Dong Li, and Ignacio Laguna. 2021. Paris: Predicting application resilience using machine learning. *J. Parallel and Distrib. Comput.* 152 (2021), 111–124.

[19] Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. 2021. Transformer in transformer. *Advances in neural information processing systems* 34 (2021), 15908–15919.

[20] Qinghao Hu, Peng Sun, Shengen Yan, Yonggang Wen, and Tianwei Zhang. 2021. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[21] Tyler Hunt, Zhipeng Jia, Vance Miller, Christopher J Rossbach, and Emmett Witchel. 2019. Isolation and beyond: Challenges for system security. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. 96–104.

[22] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. 2023. Fast and efficient model serving using multi-GPUs with direct-host-access. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 249–265.

[23] Zhuoran Ji and Cho-Li Wang. 2022. Compiler-Directed Incremental Checkpointing for Low Latency GPU Preemption. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 751–761.

[24] Chenyu Jiang, Zhen Jia, Shuai Zheng, Yida Wang, and Chuan Wu. 2024. DynaPipe: Optimizing multi-task training through dynamic pipelines. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 542–559.

[25] Ouwen Jin, Qinghui Xing, Ying Li, Shuiguang Deng, Shuibing He, and Gang Pan. 2023. Mapping very large scale spiking neuron network to neuromorphic hardware. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 419–432.

[26] Nikhil Ketkar, Jojo Moolayil, Nikhil Ketkar, and Jojo Moolayil. 2021. Introduction to pytorch. *Deep learning with python: learn best practices of deep learning models with PyTorch* (2021), 27–91.

[27] Rudolf Kruse, Sanaz Mostaghim, Christian Borgelt, Christian Braune, and Matthias Steinbrecher. 2022. Multi-layer perceptrons. In *Computational intelligence: a methodological introduction*. Springer, 53–124.

[28] Oh-Kyoung Kwon, Ji-hoon Kang, Seungchul Lee, Wonjung Kim, and Junehwa Song. 2022. Efficient task-mapping of parallel applications using a space-filling curve. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. 384–397.

[29] Baolin Li, Viiay Gadepally, Siddharth Samsi, and Devesh Tiwari. 2022. Characterizing multi-instance gpu for machine learning workloads. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 724–731.

[30] Zichong Li, Lan Zhang, Mu Yuan, Miaohui Song, and Qi Song. 2023. Efficient deep ensemble inference via query difficulty-dependent task scheduling. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 1005–1018.

[31] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.

[32] Chengzhi Lu, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2023. Understanding and optimizing workloads for unified resource management in large cloud platforms. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 416–432.

[33] Donald W Marquardt and Ronald D Snee. 1975. Ridge regression in practice. *The American Statistician* 29, 1 (1975), 3–20.

[34] McKinsey & Company. 2024. AI power: Expanding data center capacity to meet growing demand. Online. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/ai-power-expanding-data-center-capacity-to-meet-growing-demand Accessed: 2025-03-19.

[35] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. 2022. Looking beyond GPUs for DNN scheduling on Multi-Tenant clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 579–596.

[36] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging AI applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.

[37] Kelvin KW Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 595–610.

[38] Chandandeep Singh Pabla. 2009. Completely fair scheduler. *Linux Journal* 2009, 184 (2009), 4.

[39] Bo Pang, Erik Nijkamp, and Ying Nian Wu. 2020. Deep learning with tensorflow: A review. *Journal of Educational and Behavioral Statistics* 45, 2 (2020), 227–248.

[40] David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David R So, Maud Texier, and Jeff Dean. 2022. The carbon footprint of machine learning training will plateau, then shrink. *Computer* 55, 7 (2022), 18–28.

[41] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. CASSINI:Network-Aware Job Scheduling in Machine Learning Clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.

[42] Zeineb Rejiba and Javad Chamanara. 2022. Custom scheduling in kubernetes: A survey on common problems and solution approaches. *Comput. Surveys* 55, 7 (2022), 1–37.

[43] Hans Sagan. 2012. *Space-filling curves*. Springer Science & Business Media.

[44] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. *Advances in neural information processing systems* 28 (2015).

[45] Khaldoun Senjab, Sohail Abbas, Naveed Ahmed, and Atta ur Rehman Khan. 2023. A survey of Kubernetes scheduling algorithms. *Journal of Cloud Computing* 12, 1 (2023), 87.

[46] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 1075–1092.

[47] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and policy considerations for modern deep learning research. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 13693–13696.

[48] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 173–191.

[49] Shujiong Tang, Yue Yu, Hui Wang, Guiliang Wang, Wuhui Chen, Zenglin Xu, Song Guo, and Wen Gao. 2023. A survey on scheduling techniques in computing and network convergence. *IEEE Communications Surveys & Tutorials* 26, 1 (2023), 160–195.

[50] Qiange Wang, Xin Ai, Yanfeng Zhang, Jing Chen, and Ge Yu. 2023. HyTGraph: GPU-Accelerated Graph Processing with Hybrid Transfer Management. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 558–571.

[51] Yidi Wang, Mohsen Karimi, Yecheng Xiang, and Hyoseung Kim. 2021. Balancing energy efficiency and real-time performance in GPU scheduling. In *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 110–122.

[52] Sanford Weisberg. 2005. *Applied linear regression*. Vol. 528. John Wiley & Sons.

[53] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 945–960.

[54] Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yang, and Liping Zhang. 2023. Beware of fragmentation: Scheduling GPU-Sharing workloads with fragmentation gradient descent. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 995–1008.

[55] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.

[56] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. 2023. Transparent GPU sharing in container clouds for deep learning workloads. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 69–85.

[57] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. 2018. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 595–610.

[58] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. 2020. AntMan: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 533–548.

[59] Kaiqiang Xu, Decang Sun, Han Tian, Junxue Zhang, and Kai Chen. 2025. GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 999–1014.

[60] Kaiqiang Xu, Decang Sun, Hao Wang, Zhenghang Ren, Xinchen Wan, Xudong Liao, Zilong Wang, Junxue Zhang, and Kai Chen. 2025. Design and Operation of Shared Machine Learning Clusters on Campus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 295–310.

[61] Pan Xu, Cuong Nguyen, and Srikanta Tirthapura. 2018. Onion curve: A space filling curve with near-optimal clustering. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 1236–1239.

[62] Neeraja J Yadwadkar, Francisco Romero, Qian Li, and Christos Kozyrakis. 2019. A case for managed and model-less inference serving. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. 184–191.

[63] Zhisheng Ye, Wei Gao, Qinghao Hu, Peng Sun, Xiaolin Wang, Yingwei Luo, Tianwei Zhang, and Yonggang Wen. 2024. Deep learning workload scheduling in gpu datacenters: A survey. *Comput. Surveys* 56, 6 (2024), 1–38.

[64] Zhisheng Ye, Peng Sun, Wei Gao, Tianwei Zhang, Xiaolin Wang, Shengen Yan, and Yingwei Luo. 2021. Astraea: A fair deep learning scheduler for multi-tenant gpu clusters. *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2021), 2781–2793.

[65] Xinchun Zhang, Aqsa Kashaf, Yihan Zou, Wei Zhang, Weibo Liao, Haoxiang Song, Jintao Ye, Yakun Li, Rui Shi, Yong Tian, et al. 2024. ResLake: Towards Minimum Job Latency and Balanced Resource Utilization in Geo-Distributed Job Scheduling. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3934–3946.

[66] Yongkang Zhang, Haoxuan Yu, Chenxia Han, Cheng Wang, Baotong Lu, Yunzhe Li, Zhifeng Jiang, Yang Li, Xiaowen Chu, and Huaicheng Li. 2025. SGDRC: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 267–281.

[67] Zhenwei Zhang, Qiang Qi, Ruitao Shang, Li Chen, and Fei Xu. 2021. Prophet: Speeding up distributed DNN training with predictable communication scheduling. In *Proceedings of the 50th International Conference on Parallel Processing*. 1–11.